FlowState: Architectural Blueprint, Technology Stack, and Execution Strategy for Agentic Urban Traffic Microsimulation

Executive Analysis of the FlowState Paradigm

The conceptualization of FlowState—a data-driven, real-time microsimulation and control platform aimed at mitigating urban traffic bottlenecks—addresses one of the most persistent, economically damaging, and environmentally hazardous inefficiencies in modern metropolitan infrastructure. Urban traffic congestion at bottlenecks, such as lane reductions, toll plazas, or railway crossings, is fundamentally a non-linear, multi-agent behavioral phenomenon governed by spatial constraints. When wide multi-lane corridors narrow, the breakdown of structured lane discipline and the introduction of aggressive merging maneuvers result in stop-and-go shockwave traffic jams, which drastically reduce vehicular throughput and cascade delays backward through the transportation network.

The scope of the FlowState project is exceptionally well-aligned with the demands of a Machine Learning Engineering track at a competitive hackathon. It effectively combines physics-based microscopic traffic simulation with advanced reinforcement learning algorithms and multi-agent systems driven by large language models. The core inefficiencies identified in the original proposition—the capacity drop, diagonal conflicts, and stop-and-go shockwaves—are well-documented empirical phenomena in traffic flow theory.[1] By creating a live, interactive digital twin of a specific urban bottleneck and deploying dynamic speed harmonization (DSH) via machine learning, FlowState proposes a viable, scalable, and mathematically rigorous intervention. However, to elevate the project from a standard cellular automata demonstration to a production-grade engineering solution, several strategic additions, architectural refinements, and algorithmic enhancements must be integrated into the core framework.

Empirical Context: The Urban Congestion Crisis

To establish the necessity of the FlowState platform, the simulation must be grounded in empirical realities rather than abstract environments. The magnitude of urban congestion is particularly evident in high-density metropolitan environments. Recent mobility data underscores the severity of this crisis. In the year 2025, India emerged as the fifth most traffic-congested country globally and the second most congested in Asia.[2] The analysis of specific metropolitan areas reveals staggering losses in economic productivity and personal time. New

Delhi, for instance, was ranked as the third most congested metro city in India and the seventh in Asia, with commuters losing an average of 104 hours annually to traffic congestion, representing a sharp rise from previous years.[2]

During evening peak hours in New Delhi, the average travel time for a 10-kilometer trip expands to 31 minutes and 45 seconds, with congestion levels surging to 107.4% and average vehicle speeds dropping to a mere 18.9 kmph.[2] The data indicates that congestion is consistently worst between the hours of 4:00 PM and 6:00 PM, exceeding 100% across both weekdays and weekends.[2] Other major urban centers exhibit similar or worse metrics; Bengaluru ranks as the second most congested city globally, with an average speed of 13.9 kmph during peak hours and an annual loss of 168 hours per commuter.[4]

Metropolitan Region	**Global Rank (2025)**	**Asian Rank (2025)**	**Avg. Speed Peak (kmph)**	**Annual Time Lost (Hours)**	**Congestion Level**
Bengaluru, India	2	1	13.9	168	74.4%
Pune, India	5	2	18.0	152	71.1%
Mumbai, India	18	6	20.8	126	63.2%
New Delhi, India	23	7	18.9	104	60.2%
Kolkata, India	29	9	17.4	115 (approx)	58.0% (approx)

Table 1: Comparative analysis of traffic congestion metrics across major Indian metropolitan areas based on 2025 location technology data.[2]

Grounding the FlowState simulation in a specific, notoriously congested real-world location provides a highly compelling narrative for a hackathon presentation. The simulation environment should be modeled after the Ashram Chowk to Badarpur border stretch on Mathura Road (NH2) in New Delhi.[9] This 8-kilometer corridor is heavily utilized, carrying over 400,000 vehicles during peak hours, and routinely forces commuters to endure 45-minute travel times for a journey that should theoretically take 15 minutes.[10] The bottleneck is characterized by flawed geometric design, abrupt lane reductions, and merging traffic from adjacent regions like Noida and Faridabad.[9] Modeling this specific geometry—including the four-lane Ashram underpass [11]—will validate the dynamic control policy against highly complex, asymmetric merging conditions. Alternative locations for simulation modeling include the Dhaula Kuan to Airport road, which suffers from severe narrowing due to metro pillar infrastructure [12], or the Mukarba Chowk underpass, where service road merging creates distinct weaving conflicts.[14]

Mathematical and Theoretical Formulation of Bottleneck Dynamics

The fidelity of the FlowState microsimulation relies entirely on the mathematical models governing vehicle kinematics and the macroscopic flow phenomena that emerge from these individual interactions.

The Phenomenon of the Capacity Drop

The macroscopic effect of an urban bottleneck is mathematically described by the capacity drop phenomenon. In fundamental traffic flow theory, the capacity of a transportation corridor can be simplified as a function of the number of people transported, the velocity of the transport, and the spatial area consumed. A generalized representation defines capacity as the product of the number of people per vehicle and vehicle velocity, divided by the spatial area taken up.[17] However, in real-world highway capacity manuals, the theoretical free flow speed is reduced by adjustment factors for lane width, lateral clearance, and median types.[18]

Empirical observations across various freeway bottlenecks demonstrate a distinct relation between vehicle density and losses in discharge flow.[1] As vehicles approach a constraint, the density measured over the extended-length freeway segment increases. Once queues form just upstream of the constraint, the bottleneck suffers a pronounced reduction in discharge flow, a phenomenon that is highly reproducible across different geometries.[1] This capacity drop occurs because the aggressive merging maneuvers force following vehicles to decelerate sharply, creating a localized drop in speed that propagates backward as a stop-

and-go shockwave.[19] The primary objective of the FlowState system is to utilize dynamic control logic to prevent the density from reaching the critical threshold that triggers this capacity drop.[1]

The Intelligent Driver Model (IDM)

To simulate longitudinal vehicle dynamics—specifically car-following behavior—FlowState must abandon rudimentary cellular automata in favor of the Intelligent Driver Model (IDM). The IDM is widely recognized as the industry standard due to its continuous, collision-free differential equations that smoothly balance non-interaction acceleration with interaction-based braking.[21] The acceleration of a given vehicle is calculated based on its current speed, the distance to the leading vehicle, and the speed difference between the two. The fundamental equation governing the acceleration rate is defined by a ratio of the current speed to the desired speed, combined with a ratio of the desired minimum gap to the actual gap.[21]

The desired minimum gap itself is a dynamic variable. It is calculated as the sum of a static minimum distance, a time headway component dependent on current velocity, and an intelligent braking strategy term that considers the velocity difference and the vehicle's comfortable acceleration and deceleration parameters.[21] By manipulating these parameters, the simulation can accurately reflect distinct aspects of driving behavior, ranging from timid to aggressive, or from anticipative to short-sighted.[21]

Persona Classification	**Desired Speed**	**Time Headway**	**Minimum Gap**	**Max Acceleration**	**Comfortable Deceleration**
Passive Passenger Car	80 km/h	1.8 s	3.0 m	0.8 m/s^2	1.5 m/s^2
Aggressive Passenger Car	120 km/h	0.8 s	1.0 m	2.0 m/s^2	3.0 m/s^2

Standard Heavy Truck	80 km/h	1.7 s	2.0 m	0.3 m/s^2	2.0 m/s^2

Table 2: Parametric configuration of the Intelligent Driver Model for distinct agent personas, calibrated for realistic variance in heterogeneous urban traffic streams.[23]

By assigning these distinct parametric profiles to the simulated fleet, FlowState ensures that the traffic stream exhibits the heterogeneous, unstructured behaviors typical of junctions in New Delhi. The interaction between passive vehicles yielding early and aggressive vehicles executing late diagonal merges creates the precise conditions necessary for shockwave formation.

The FlowState Microsimulation Engine Architecture

Building a high-performance microsimulation engine is a critical technical challenge. While established open-source traffic simulators exist, evaluating them against the specific constraints of a 48-hour hackathon reveals distinct trade-offs.

Evaluating Existing Simulation Frameworks

Eclipse SUMO (Simulation of Urban Mobility) is a highly robust, continuous multi-modal traffic simulation package developed in C++ and Python.[26] It offers comprehensive tools for network importing, route finding, and emission calculations.[28] However, SUMO's architecture, which relies on the Traffic Control Interface (TraCI) for external script communication, can introduce significant overhead when attempting to stream high-frequency, 60 frames-per-second coordinate updates to a web-based visualization dashboard.[26]

CityFlow is an alternative open-source traffic simulator designed specifically for large-scale multi-agent reinforcement learning. It boasts execution speeds more than twenty times faster than SUMO and supports city-wide simulations with tens of thousands of vehicles.[29] CityFlow provides a user-friendly Python interface for controlling traffic signals and retrieving state information.[29] Other alternatives include UXsim, a lightweight, pure Python macroscopic and mesoscopic network traffic flow simulator [30], and OpenTrafficSim, a Java-based project combining micro and macro simulation.[31]

Despite the power of these existing tools, integrating them into a custom, visually striking React dashboard within a hackathon timeframe often results in complex dependency hell and cross-compilation errors. For the FlowState project, constructing a bespoke, lightweight 2D physics engine in C++ is highly recommended.

Custom C++ Engine Implementation

A custom C++ engine guarantees raw multi-agent execution speed and allows for precise memory management when iterating the IDM equations over hundreds of vehicles. To handle the geometric complexities of the road network and prevent overlapping vehicles, the engine should leverage a header-only 2D collision detection library. Libraries such as tinyc2 or Physac provide robust, fast boolean result functions for detecting intersections between bounding boxes, circles, and convex polygons.[32] tinyc2, implemented in portable C, features generic collision functions and manifold generation routines that perfectly suit the needs of a top-down traffic simulation without the bloat of a full commercial game engine.[33]

Within this custom engine, lane-changing logic must be explicitly modeled. Vehicles require mechanisms to evaluate adjacent lanes for acceptable gaps before executing lateral movements.[34] The engine must implement parameters equivalent to the Lane Change Distance and Emergency Stop Distance utilized in commercial software like VISSIM. The Lane Change Distance defines the point upstream of a connector where a vehicle will first attempt a required lane change, while the Emergency Stop Distance defines the location where a vehicle will completely halt if an acceptable gap is unavailable, thereby triggering a localized bottleneck.[35]

Integration of Generative Traffic Agents (GTA)

While the Intelligent Driver Model and probabilistic lane-changing algorithms simulate kinematic physics accurately, they rely on handcrafted rules that struggle to capture the nuanced, context-dependent nature of human decision-making. To achieve a higher degree of realism, FlowState must integrate Large Language Models (LLMs) to serve as Generative Traffic Agents (GTA) for a subset of the simulated vehicles.

The integration of LLMs into traffic simulation represents the cutting edge of transportation research. Generative Traffic Agents simulate large-scale, context-sensitive transportation choices by employing persona-based agents grounded in sociodemographic data.[37] Modern GTA frameworks utilize multi-tensor representations of the simulated world and employ diffusion processes to generate plausible future maneuvers.[39] By replacing fixed decision rules with LLM personas, the simulation can emulate human-like routing, mode choice, and micro-decision-making without requiring complex, hand-coded heuristics.[37]

In the FlowState architecture, an LLM agent operates through a continuous cycle of perception, reasoning, decision-making, execution, and feedback.[40] When an agent approaches a bottleneck, the FastAPI orchestration layer constructs a contextual prompt describing the vehicle's current speed, its distance to the narrowing lane, the behavior of surrounding vehicles, and the agent's specific persona description (e.g., a delayed commuter in a heavy commercial vehicle). The LLM processes this spatial and temporal context and

outputs an intended action, such as a decision to force a merge or yield to an aggressive driver.[41]

However, deploying LLM agents at scale presents challenges. Current GTA frameworks exhibit limitations related to inference latency, API costs, and inherent model biases.[39] LLMs frequently exhibit a role-model effect, where they over-select socially desirable behaviors and under-represent aggressive or erratic maneuvers.[39] To mitigate this latency and bias within the context of a hackathon, the LLM-driven generative agents should be restricted to a small percentage (approximately 5% to 10%) of the total fleet. These entities act as highly realistic chaos agents, introducing unpredictable, context-aware human errors that disrupt the uniform flow maintained by the deterministic IDM vehicles.

Analytical Observability via LangChain

To fulfill the stretch goal of an AI analytical copilot, the project must implement a robust observability pipeline. LangChain and LangSmith provide the necessary infrastructure to manage these LLM interactions.[45] By capturing the simulation telemetry logs—including timestamps of shockwave formations, capacity drop events, and the internal reasoning trajectories of the GTA vehicles—the system builds a dynamic vector store.[47]

A LangChain-powered retrieval-augmented generation (RAG) agent interfaces with the React dashboard. When hackathon judges ask complex analytical questions, such as inquiring why throughput dropped precipitously at a specific minute, the LangChain agent queries the vector store, analyzes the sequence of aggressive lane changes that precipitated the bottleneck, and returns a natural language diagnosis.[45] LangSmith acts as the observability platform, tracing the agent's logic, monitoring token costs, and debugging latency spikes, which provides undeniable proof of the system's technical sophistication.[46]

Reinforcement Learning and Dynamic Speed Harmonization

The core objective of FlowState is the mitigation of bottleneck congestion through an adaptive machine learning control strategy. The chosen methodology is Dynamic Speed Harmonization (DSH), an approach that regulates the speed of vehicles approaching a constrained roadway segment.[49] By dynamically lowering the speed limit of incoming traffic when a shockwave is detected, the algorithm dampens oscillations, prevents vehicles from arriving at the bottleneck simultaneously, and smooths the overall traffic flow.[50]

Reinforcement Learning Architecture

The optimization of speed harmonization in non-linear, mixed-autonomy traffic flows is perfectly suited for Deep Reinforcement Learning (DRL).[50] Unlike traditional heuristic

algorithms or fixed-time control methods, DRL agents continuously interact with the environment, adapting their strategies based on real-time traffic conditions.[51]

Within the realm of DRL, various algorithms have been evaluated for traffic control. Deep Q-Networks (DQN) utilize discrete action spaces and value-iteration, while Advantage Actor-Critic (A2C) methods update policy and value functions simultaneously.[53] However, Proximal Policy Optimization (PPO) has emerged as the superior algorithm due to its stable learning curve, sampling efficiency, and robustness in continuous environments.[53] PPO employs a clipping mechanism in its objective function to prevent destructively large policy updates, making it highly effective for learning policies that maximize long-term rewards in highly stochastic traffic scenarios.[53]

The mathematical formulation of the PPO environment in FlowState is defined as follows:

- **State Space Formulation:** The observation space must capture the spatiotemporal dynamics of the bottleneck.[56] The state vector includes the average velocity of approaching vehicles, the real-time density matrix of the road segments, and the calculated length of the backward-propagating shockwave. Advanced implementations may utilize an autoencoder representation to compress the highly stochastic state space into a lower-dimensional embedding for the RL controller.[53]
- **Action Space Formulation:** The action space represents the dynamic speed limit broadcast to the approaching vehicles via simulated variable message signs.[57] The PPO agent outputs a continuous multiplier that scales the desired speed parameter of the IDM for vehicles within the control zone.
- **Reward Function Engineering:** The design of the reward function is the most critical component of the DRL architecture, as it explicitly defines the agent's objective.[53] A poorly formulated reward can lead to unintended behaviors, such as the agent stopping all traffic to prevent merging conflicts. The primary objective is to minimize the total accumulated waiting time across all vehicles. This can be expressed as a penalty function where the reward is the negative sum of the waiting time of all vehicles in the network:

$$R_t = -\sum_{i=1}^{N} w_i(t)$$

Where $w_i(t)$ denotes the waiting time of vehicle i at time step t.[59] To encourage stable throughput optimization and reduce variance during training, the reward can alternatively be formulated as the difference in accumulated waiting times between consecutive control cycles:

$$R_t = \sum_{i=1}^{n} w_{t-1}^{i} - \sum_{j=1}^{m} w_t^{j}$$

Where w_{t-1}^{i} is the waiting time of the i-th vehicle in the previous cycle, and w_t^{j} is the waiting time of the j-th vehicle in the current cycle.[58] By maximizing this reward, the PPO agent learns to prioritize actions that keep vehicles moving continuously, thereby neutralizing the capacity drop.[53]

Algorithm Type	**Action Space**	**Training Stability**	**Application Suitability for Traffic**
Deep Q-Network (DQN)	Discrete	Moderate (requires target networks)	Suitable for simple phase switching
Advantage Actor-Critic (A2C)	Continuous/Discrete	Low to Moderate	Struggles with high-variance environments
Proximal Policy Optimization (PPO)	Continuous	High (clipping mechanism)	Optimal for dynamic speed harmonization

Table 3: Comparative evaluation of Deep Reinforcement Learning algorithms for traffic control optimization.[53]

Environmental Modeling and Emergency Vehicle Preemption

To demonstrate a comprehensive understanding of intelligent transportation systems, FlowState must model the secondary impacts of traffic bottlenecks, specifically vehicular emissions and the disruption caused by emergency responders.

Microscopic Emission Modeling

Road traffic is a massive contributor to global greenhouse gas emissions, and stop-and-go congestion exacerbates fuel consumption and pollutant release exponentially.[61] Accurately estimating these emissions requires integrating an emission model with the microscopic traffic simulator.

Macroscopic models, such as the Handbook of Emission Factors for Road Transport (HBEFA), provide estimates based on average driving conditions but lack the granularity required to capture the spikes in emissions caused by aggressive acceleration during shockwave recovery.[63] Conversely, microscopic models like the Comprehensive Modal Emission Model (CMEM) calculate fuel consumption and CO2 emissions on a second-by-second basis, utilizing the instantaneous speed and acceleration data generated by the IDM equations.[64] By integrating a simplified CMEM formulation, the FlowState dashboard can visualize the real-time reduction in carbon emissions achieved when the PPO agent smooths the traffic flow, providing a powerful secondary metric for evaluation.

Emergency Vehicle Priority and Dilemma Zones

The robustness of the machine learning controller is truly tested when unpredictable disruptions occur. The introduction of an emergency vehicle, such as an ambulance or fire truck, fundamentally alters the traffic dynamics. In reality, emergency vehicle prioritization is achieved through move-over laws and traffic signal preemption, where the vehicle requests a green signal via acoustic sensors, optical emitters, or connected vehicle technology.[66]

When an emergency vehicle enters the FlowState network, the microsimulation engine must execute complex right-of-way logic. Non-priority vehicles ahead of the emergency vehicle must dramatically alter their lane-changing parameters to yield, effectively creating a temporary, moving bottleneck.[35] This introduces a "dilemma zone" where vehicles must rapidly decide whether to accelerate to clear the path or brake to yield.[67] The sudden disruption forces the PPO agent to rapidly adapt its speed harmonization strategy to clear the subsequent queue and restore optimal throughput after the emergency vehicle has passed.[67]

Communication Protocols: V2X, WebSockets, and WebAssembly

The mechanism through which the reinforcement learning algorithm influences the vehicles must be explicitly modeled to reflect the realities of modern smart city infrastructure. Instead of omnisciently altering vehicle speeds, FlowState simulates Vehicle-to-Everything (V2X) communication.

Modeling V2X Penetration

V2X technologies enable vehicles to exchange Cooperative Awareness Messages over Dedicated Short-Range Communications (DSRC) or Cellular V2X (C-V2X) networks.[71] Simulating this communication layer introduces realistic constraints, such as signal attenuation and data latency, which significantly impact the efficacy of speed harmonization.[71]

Furthermore, evaluating the system under varying V2X penetration rates provides critical insights into the transitional phase of autonomous driving. By simulating scenarios with 0%, 50%, and 100% V2X adoption, the project demonstrates how the dynamic gating algorithm performs in mixed-autonomy traffic.[73] At partial adoption levels, the PPO agent must rely on the connected vehicles to act as platoon leaders, pacing the unconnected, manually driven vehicles trailing behind them.[57]

Real-Time Streaming Architecture

To transmit the high-frequency state matrix from the backend simulation engine to the frontend visualization dashboard, a robust real-time communication protocol is required.

WebAssembly (WASM) presents one potential pathway. Compiling the C++ simulation engine to WebAssembly using Emscripten allows the heavy physics calculations to execute directly within the client's browser, eliminating network latency.[75] Benchmarks indicate that WASM significantly outperforms JavaScript for computational tasks, yielding speedups ranging from 2x to 30x depending on the browser environment.[76] The generated WebAssembly runs inside a WebWorker, allowing the React UI thread to remain responsive while rendering the entities.[75] However, executing the simulation client-side complicates the integration of the Python-based PPO reinforcement learning model, which requires heavy server-side processing.

Therefore, a server-side execution model utilizing WebSockets or Socket.IO is the optimal architectural choice. Raw WebSockets provide a persistent, full-duplex TCP connection with minimal framing overhead, offering the lowest possible latency for high-throughput coordinate streaming.[79] However, raw WebSockets require the developer to manually implement reconnection logic, heartbeat mechanisms, and message buffering.[82]

Socket.IO, built on top of WebSockets, abstracts these complexities by providing automatic reconnections with exponential backoff, room management, and fallback transports like HTTP long-polling.[79] While Socket.IO introduces slight metadata overhead to each message, the developer experience it provides is invaluable during a time-constrained hackathon.[82] The availability of a robust C++ client library (socket.io-client-cpp) ensures that the core physics engine can effortlessly emit binary state data to the orchestration layer.[85]

Communication	**Latency**	**Overhead**	**Reliability**	**Hackathon**

Protocol			Features	Suitability
WebAssembly (Client-Side)	Zero (Local)	None	N/A (Runs in browser)	Low (Complex ML integration)
Raw WebSockets	Extremely Low	Minimal	Manual implementation required	Moderate
Socket.IO	Low	Moderate (Metadata)	Auto-reconnect, fallbacks	High (Rapid prototyping)

Table 4: Analysis of real-time communication and execution protocols for streaming simulation telemetry.[77]

The FastAPI Orchestration Bridge

To manage the complex flow of data between the C++ engine, the Python PPO model, and the React frontend, FastAPI serves as the central orchestration bridge. FastAPI's native support for asynchronous programming (asyncio) and WebSockets makes it exceptionally well-suited to handle concurrent data streams.[87] The C++ engine emits the vehicle coordinate arrays to a FastAPI WebSocket endpoint. Simultaneously, the PPO agent evaluates the incoming state data, calculates the optimal speed limits, and injects these commands back into the C++ stream. Finally, FastAPI broadcasts the unified state payload—comprising vehicle positions, dynamic speed limits, and calculated telemetry metrics—to the React frontend for visualization.[89]

Monorepo Organization and Build Toolchain

Organizing a project with distinct C++, Python, and React components requires a disciplined file structure. A monorepo architecture managed by pnpm workspaces is the standard best practice for such multi-language deployments.[92] A monorepo centralizes configuration files, simplifies dependency management, and allows developers to orchestrate the entire

application stack from a single root directory.[94]

The React frontend should be initialized using Vite, a build tool that provides ultra-fast Hot Module Replacement (HMR) and optimized bundling.[97] The C++ engine utilizes CMake for cross-platform compilation, ensuring that the physics logic and the Socket.IO client library build consistently across different operating systems.[96]

The file structure is organized to separate the rendering layer, the API gateway, and the core simulation logic:

```
flowstate-monorepo/
├── pnpm-workspace.yaml
├── package.json
├──.gitignore
│
├── apps/
│   ├── web-dashboard/
│   │   ├── src/
│   │   │   ├── components/
│   │   │   ├── canvas/
│   │   │   ├── App.tsx
│   │   │   └── main.tsx
│   │   ├── tailwind.config.js
│   │   ├── vite.config.ts
│   │   └── package.json
│   │
│   ├── api-gateway/
│   │   ├── src/
│   │   │   ├── ml_models/
│   │   │   ├── llm_copilot/
│   │   │   ├── routers/
```

```
│   │   │   └── main.py
│   │   ├── requirements.txt
│   │   └── Dockerfile
│
├── packages/
│   ├── engine-cpp/
│   │   ├── include/
│   │   ├── src/
│   │   │   ├── PhysicsWorld.cpp
│   │   │   ├── VehicleAgent.cpp
│   │   │   └── SocketIOClient.cpp
│   │   ├── CMakeLists.txt
│   │   └── Makefile
│   │
│   ├── shared-types/
│   │   ├── index.ts
│   │   └── package.json
```

Evaluation Metrics and Real-World Applicability

The ultimate success of the FlowState platform relies on its ability to demonstrably improve predefined evaluation metrics. The command dashboard provides live telemetry proving the effectiveness of the control strategy.

The primary success metric is Throughput, measured in vehicles per minute passing the bottleneck constraint. Under baseline conditions, throughput drops significantly as the shockwave forms.[1] The dynamic speed harmonization control is expected to increase sustained throughput by preventing full-stop gridlocks.[60] Average Waiting Time tracks the delay of each agent compared to an unobstructed journey, directly reflecting the economic time lost by commuters.[2] Congestion Length visualizes the backward propagation of the traffic jam. Furthermore, integrating the CMEM emission logic calculates the reduction in CO2

output resulting from smoothed acceleration profiles.[64]

While FlowState is a simulated environment, the architecture and the control logic are directly applicable to the physical infrastructure of smart cities. By utilizing data from overhead camera feeds, radar, or smart-toll systems to populate the state matrix, urban transportation authorities could deploy FlowState's dynamic gating algorithms in the real world. The calculated speed limits would be transmitted via Variable Message Signs positioned upstream of critical intersections, or broadcast directly to connected vehicles via C-V2X networks.[57] The optimization of these bottlenecks translates directly to reduced commute times, lower idle carbon emissions, and a decrease in the likelihood of minor collisions caused by aggressive lane-changing maneuvers, thereby addressing one of the most critical challenges in modern urban planning.